DevOps

Projektbezeichnung	Sprachsteuerung der Spritzgussmaschine
Projektverantwortlicher	Armin Anton
Verantwortlicher DevOps	Armin Anton
Erstellt am	24.06.2025
Zuletzt geändert	24.06.2025
Bearbeitungszustand	In Bearbeitung
Dokumentbezeichnung	DevOps.docx

Historie der Dokumentversionen

Version	Datum	Autor	Änderungen/Bemerkungen
0.1	24.06.2025	Armin Anton	Ersterstellung
0.2			
0.3			
0.4			

Diese Dokumentation beschreibt die DevOps-Strategie, -Tools und -Prozesse, die im Rahmen dieses Projekts verwendet werden. Sie dient als zentrale Referenz für alle Beteiligten und soll Transparenz, Nachvollziehbarkeit sowie eine gemeinsame technische Basis schaffen.

Libraries:

Folgende Libraries sind im Projekt zur verwendung gekommen

c#:	**python:**
NAudio	os
Accord.Math	numpy
FftSharp	librosa
MathNet.Numerics	soundfile
	scipy

Dokumente:

Zum Erstellen der Dokumente werden Word oder Google Docs verwendet, je nach Präferenz. Beide werden als .docx abgespeichert.

Discord:

Zur effizienten und teamübergreifenden Kommunikation nutzen wir einen dedizierten Discord-Server. Dieser dient als zentrale Plattform für den Austausch zwischen Teammitgliedern. Ziel ist es, schnelle Abstimmungen zu ermöglichen, technische Fragen zügig zu klären und relevante Informationen transparent zu teilen.

Mit Strukturierten Sprach und Text Kanälen wird die Kommunikation thematisch gegliedert und sorgt dafür, dass wir immer beim Thema bleiben.

Sprach Calls mit Screensharing sorgen für eine zügige Verständigung und Problembehandlung.

Git:

Git ist unser zentrales Versionskontrollsystem zur Verwaltung des Quellcodes und der Dokumentation. Es ermöglicht verteilte Entwicklung, Nachvollziehbarkeit von Änderungen und sicheres Arbeiten an parallelen Features.

In unserem Fall halten wir die Nutzung von Git simpel und benutzen nur den Main Branch.

Visual Studio:

Unsere Anwendungen werden in C# auf Basis von .NET entwickelt. Als primäre Entwicklungsumgebung verwenden wir Visual Studio, da es eine tiefe Integration in das .NET-Ökosystem, leistungsstarke Debugging-Tools sowie umfangreiche Unterstützung für Unit-Testing und Git bietet.

Die Solution ist in mehrere Projekte aufgeteilt, um Logik und User Interface auseinanderzuhalten.

Die Gui hat sein eigenes Projekt, während die Logik in einer Class Library ist.

Audio Samples:

Um Audio samples effizient zu vergleichen, müssen diese konvertiert werden.

Um die Implementierung zu vereinfachen, wurde dieser Aspekt in Python realisiert. Alle .wav Audio Dateien werden mit einem separaten python script zum mfcc format in .csv Dateien gespeichert. Diese .csv Dateien können dann von der Logik aufgerufen werden.

Das Format für die .wav und .csv dateien ist wie folgt:

Name{variation-nr}_{Sprachbefehl}.wav

Armin1_5.wav -> der erste sprachbefehl “fünf”

Armin3_5.wav -> die dritte variation vom sprachbefehl “fünf”

GanttProject:

Für die zeitliche Planung und Visualisierung unseres Projekts verwenden wir GanttProject, ein Open-Source-Tool zur Erstellung von Gantt-Diagrammen. Jeder Projektphase wurde eine klare Deadline zugewiesen, um einen zielgerichteten und strukturierten Projektverlauf sicherzustellen.

Draw.io:

Zur Visualisierung der Softwarearchitektur und insbesondere der Klassenstrukturen verwenden wir draw.io. Das Tool bietet eine einfache und intuitive Oberfläche zur Erstellung von Diagrammen und eignet sich ideal für die Darstellung objektorientierter Strukturen. In unserem Fall wurde es genutzt, um ein übersichtliches Klassendiagramm zu entwerfen, das die Beziehungen zwischen den wichtigsten Komponenten unserer C#-Anwendung abbildet.

Benutzung der GUI:

Die Benutzung der Gui ist sehr einfach gestaltet. Als User muss man nach Start des Programms einfach auf den Start-Knopf klicken. Nun ist das Mikrofon aktiv und es werden in einem unteren Fenster die Audiodaten als Zeitsignal gezeigt.

Oben links wird der erkannte Befehl angezeigt, während rechts eine Historie von vorherigen Befehlen angezeigt wird.

